

Quantifying and Reducing Semantic Drift in Text-to-SQL Systems: A Constraint-Driven Multi-Agent Approach

K Sai Chetan*, Tharun G*, Tanish S[†], Gayithri BR[‡], Prof. Deepthi L[§], Prof. Mekhala Vinod Purohit[§], Dr. Somesh Nandi[‡]

*Department of Computer Science and Engineering, Ramaiah Institute of Technology, Bengaluru, India

Email: {1rv23cd404, 1rv23cd405}@rit.edu.in

[†]Department of Artificial Intelligence, Ramaiah Institute of Technology, Bengaluru, India

Email: 1rv22ai060@rit.edu.in

[‡]Department of AI and ML Engineering, Ramaiah Institute of Technology, Bengaluru, India

Email: {1rv23ai401, somesh.nandi}@rit.edu.in

[§]Department of Computer Science and Engineering, Ramaiah Institute of Technology, Bengaluru, India

Email: {deepthi.l, mekhala.vinod}@rit.edu.in

Abstract—This paper presents a novel constraint-driven multi-agent architecture for quantifying and reducing semantic drift in text-to-SQL systems. We introduce a composite semantic drift metric that evaluates query fidelity across three dimensions: intent alignment (40% weight), constraint adherence (30% weight), and result plausibility (30% weight). The system employs five specialized agents—Intent Parser, Ontology Mapper, Constraint Validator, Execution Planner, and Result Verifier—that iteratively refine natural language queries until convergence below a configurable threshold (default: 0.15). Our approach integrates a formal business ontology using NetworkX property graphs and enforces enterprise governance rules through a critic loop architecture. Experimental evaluation on synthetic datasets demonstrates significant improvements in semantic accuracy, with the drift metric providing objective convergence criteria. The implementation achieves production-grade reliability with comprehensive unit testing (40+ tests) and supports both REST API and interactive dashboard interfaces.

Index Terms—semantic drift, text-to-SQL, multi-agent systems, constraint validation, business ontology, natural language interfaces, drift detection, governance compliance

I. INTRODUCTION

Natural language interfaces to databases (NLIDB) have emerged as critical tools for democratizing data access in enterprise environments. However, existing text-to-SQL systems suffer from semantic drift, where generated queries progressively diverge from user intent due to lexical ambiguity, ontological mismatches, and constraint violations. This drift manifests as cascading failures across the transformation pipeline, resulting in queries that execute successfully but return semantically incorrect data.

Traditional approaches rely on end-to-end neural models or rule-based parsers, but these often lack the governance controls required for enterprise deployments. Our work addresses this gap through a constraint-driven multi-agent architecture that quantifies semantic drift and enforces iterative refinement until convergence.

A. Problem Statement

Semantic drift occurs when natural language queries are mapped to database schemas without sufficient contextual anchoring. This leads to:

- **Intent Misalignment:** Cosine similarity between query embeddings and ontology paths falls below acceptable thresholds
- **Constraint Violations:** Business rules (tax exclusions, regional restrictions) are not satisfied
- **Result Anomalies:** Statistical outliers detected via z-score analysis against historical baselines

B. Contributions

Our main contributions include:

- 1) **Semantic Drift Metric:** A composite scoring mechanism combining intent alignment, constraint adherence, and result plausibility
- 2) **Critic Loop Architecture:** Iterative multi-agent refinement with convergence criteria
- 3) **Business Ontology Integration:** Formal knowledge representation using NetworkX property graphs
- 4) **Production-Grade Implementation:** Comprehensive testing, API endpoints, and deployment orchestration

II. RELATED WORK

Natural language to SQL conversion has evolved from early rule-based systems to modern neural approaches. Seq2SQL [?] and SQLNet [?] pioneered sequence-to-sequence architectures for query generation. More recent work on Spider [?] benchmark introduced complex database schemas with cross-table reasoning.

However, these approaches focus primarily on syntactic correctness rather than semantic fidelity. Research on semantic grounding [?] emphasizes ontology-aware query mapping,

Fig. 1. Overall system architecture showing the multi-agent pipeline with critic loop for drift reduction.

while constraint satisfaction problems have been explored in database query optimization [?].

Multi-agent architectures for NLIDB remain underexplored. Recent work on agentic pipelines [?] demonstrates promise in decomposing complex tasks, but lacks the governance controls needed for enterprise deployments. Our approach bridges this gap by integrating constraint validation as a first-class component with measurable drift quantification.

III. METHODOLOGY

A. System Architecture

The system employs a modular pipeline architecture with five specialized agents orchestrated through a critic loop (Figure ??). Each agent maintains typed state using Pydantic dataclasses, ensuring traceability and validation throughout the pipeline.

B. Multi-Agent Pipeline

1) *IntentParserAgent*: Extracts entities, metrics, and filters from natural language using regex patterns and confidence scoring. Supports temporal filters, numerical constraints, and alias resolution with confidence thresholds (0.85-0.98).

2) *OntologyMapperAgent*: Grounds extracted elements in a formal business ontology using NetworkX property graphs. Performs fuzzy matching with cosine similarity scoring and resolves semantic variants through synonym mapping.

3) *ConstraintValidatorAgent*: Enforces business rules including tax exclusions, regional restrictions, date range validation, and metric bounds. Each constraint returns a boolean satisfaction score.

4) *ExecutionPlannerAgent*: Generates parameterized SQL templates with optimized join strategies. Creates execution plans that abstract raw SQL while maintaining query expressiveness.

5) *ResultVerifierAgent*: Validates query outputs against statistical baselines using z-score analysis. Detects anomalies and updates historical distributions for continuous learning.

C. Semantic Drift Metric

The composite drift metric quantifies divergence across three dimensions:

$$\text{drift} = w_i(1 - a_i) + w_c(1 - a_c) + w_p(1 - a_p) \quad (1)$$

where:

- a_i : Intent alignment score (weight $w_i = 0.4$)
- a_c : Constraint adherence score (weight $w_c = 0.3$)
- a_p : Result plausibility score (weight $w_p = 0.3$)

1) *Intent Alignment*: Measures semantic similarity between query embeddings and ontology paths:

$$a_i = \max_{\text{paths}} \cos(\text{query}, \text{path}) \times \left(1 - \frac{\sigma(\text{sims})}{1 + \sigma(\text{sims})}\right) \quad (2)$$

2) *Constraint Adherence*: Evaluates rule compliance as the ratio of satisfied constraints:

$$a_c = \frac{\text{satisfied}}{\text{total}} \quad (3)$$

3) *Result Plausibility*: Detects statistical anomalies using z-score analysis:

$$a_p = \frac{1}{1 + z_{\max}/\tau} \quad (4)$$

where $\tau = 3.0$ (configurable z-score threshold).

D. Critic Loop Orchestration

The LangGraph-based orchestrator implements conditional edges that route execution based on drift scores. The critic loop continues until:

$$\text{drift} < \theta \quad (\theta = 0.15 \text{ default}) \quad (5)$$

Each iteration refines the query state through agent processing, with convergence indicating semantic stability.

IV. IMPLEMENTATION

A. Technology Stack

The system is implemented in Python 3.12 with production-grade dependencies:

- **FastAPI**: REST API with automatic OpenAPI documentation
- **LangGraph**: Multi-agent orchestration framework
- **NetworkX**: Property graph representation for business ontology
- **SQLAlchemy**: Database ORM with PostgreSQL support
- **Pydantic**: Runtime type validation and serialization
- **Streamlit**: Interactive dashboard interface

B. Business Ontology

The ontology encodes domain knowledge using NetworkX property graphs with:

- **Entities**: Customer, Order, Product, Transaction, Invoice
- **Metrics**: Revenue, Profit, Order Count, Average Order Value
- **Constraints**: Tax exclusions, regional restrictions, temporal bounds
- **Relationships**: Customer-Order, Product-Order, Order-Invoice

C. Testing and Validation

Comprehensive test suite includes:

- **Unit Tests**: 40+ pytest cases covering all agents and metrics
- **Integration Tests**: End-to-end pipeline validation
- **Performance Benchmarks**: Query latency and drift convergence analysis

TABLE I
DRIFT REDUCTION PERFORMANCE ACROSS QUERY TYPES

Query Type	Initial Drift	Final Drift	Iterations
Simple Entity Match	0.12	0.08	1
Ambiguous Mapping	0.45	0.12	3
Constraint Violation	0.67	0.14	4
Complex Multi-Join	0.38	0.11	2

Fig. 2. Drift convergence across agent iterations showing progressive refinement.

V. EXPERIMENTS AND RESULTS

A. Dataset and Evaluation

Experiments use synthetic datasets with 10,500+ records across five entities. Queries test various drift scenarios including ambiguous entities, constraint violations, and statistical anomalies.

B. Drift Reduction Performance

Table ?? shows drift reduction across query categories:

C. Convergence Analysis

The critic loop demonstrates reliable convergence with 95% of queries achieving drift ≤ 0.15 within 3 iterations. Figure ?? illustrates typical convergence patterns.

D. Agent Performance Metrics

Individual agent performance:

- **IntentParserAgent:** 92% entity extraction accuracy
- **OntologyMapperAgent:** 87% semantic mapping precision
- **ConstraintValidatorAgent:** 95% rule satisfaction detection
- **ExecutionPlannerAgent:** 100% valid SQL generation
- **ResultVerifierAgent:** 89% anomaly detection accuracy

VI. DISCUSSION

A. Key Insights

The constraint-driven approach significantly improves semantic fidelity over direct text-to-SQL translation. The drift metric provides objective convergence criteria, while the multi-agent architecture enables progressive refinement.

B. Limitations

Current limitations include:

- Synthetic dataset evaluation (needs real-world validation)
- Rule-based intent parsing (could benefit from ML models)
- Static ontology (requires manual updates for new domains)

C. Enterprise Implications

The system addresses key enterprise requirements:

- **Governance Compliance:** Constraint validation prevents unauthorized data access
- **Audit Trail:** Complete query provenance and drift tracking
- **Scalability:** Modular architecture supports domain expansion

VII. CONCLUSION

We presented a constraint-driven multi-agent architecture for quantifying and reducing semantic drift in text-to-SQL systems. The composite drift metric provides objective evaluation across intent alignment, constraint adherence, and result plausibility dimensions. Experimental results demonstrate reliable convergence with significant improvements in semantic accuracy.

Future work includes integrating pre-trained language models for enhanced intent parsing, expanding ontology coverage, and deploying on real enterprise datasets. The modular design supports extension to additional domains while maintaining governance and traceability requirements.

ACKNOWLEDGMENT

The authors acknowledge the support of Ramaiah Institute of Technology for providing the research infrastructure and computational resources required for this work.

REFERENCES

- [1] Zhong *et al.*, "Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning," arXiv preprint arXiv:1709.00103, 2017.
- [2] Xu *et al.*, "SQLNet: Generating Structured Queries from Natural Language without Reinforcement Learning," arXiv preprint arXiv:1711.04436, 2017.
- [3] Yu *et al.*, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *EMNLP*, 2018, pp. 3911-3921.
- [4] Berant *et al.*, "Semantic Parsing on Freebase from Question-Answer Pairs," in *EMNLP*, 2013, pp. 1533-1544.
- [5] Kolaitis *et al.*, "Constraint Satisfaction Problems: Complexity and Approximability," in *Handbook of Computational Complexity*, 1999.
- [6] Wang *et al.*, "A Survey on Large Language Model based Autonomous Agents," arXiv preprint arXiv:2308.11421, 2023.